

ATTACHMENTS

Milestone 1: Create UI Policy on Incident

UI policies change fields on a form based on a set of conditions. Use UI policies to show or hide fields, or to make fields mandatory or read-only based on these conditions. A UI policy specifies one or more actions to take when the policy is triggered. First, create the policy. Then add as many actions as necessary. UI policy actions are applied only if all of the following conditions are met:

- The UI Policy is **Active**
- The items in the **Conditions** field evaluate to true
- The field specified in the UI policy action is present on the specified form

[More info](#)

Table: Incident [Incident] Application: Global

Active: ☒

* Short description: High Impact Control

Order: 100

When to Apply Script

Conditions: Add Filter Condition Add OR Clause

Impact is 1 - High

AND OR

If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific

Global: ☒ On load: ☒

Reverse the effects of the UI policy actions when the Conditions evaluate to false

Tables that extend the specified Table inherit this UI Policy

Reverse if false: ☒ Inherit: ☐

Milestone 2: Create UI Policy Action – Urgency

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More info](#)

UI policy: High Impact Control

Table: Incident [Incident]

* Field name: Urgency

Application: Global

Mandatory: ☒ Visible: ☒ Read only: ☒

Clear the field value: ☐

Submit

Milestone 3: Create onChange Client Script

The screenshot shows the 'Catalog Client Scripts - New Record' form in ServiceNow. The form is titled 'Catalog Client Scripts - New Record' and has a 'Submit' button. A note at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The form fields are as follows:

- Name: Auto set urgency for high impact
- Applies to: A Catalog Item
- Active: ☒
- UI Type: Desktop
- Application: Global
- Type: onChange
- Catalog Item: (empty)
- Variable name: (empty)
- Applies on a Catalog Item view: ☒
- Applies on Requested Items: ☐
- Applies on Catalog Tasks: ☐
- Applies on Target Record: ☐

The script editor contains the following code:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2
3 }
4
5 //Type appropriate comment here, and begin script below
6 function onChange(control, oldValue, newValue, isLoading) {
7   if (isLoading || newValue == '') {
8     return;
9   }
10
11   if (newValue == '1') {
12     g_form.setValue('urgency', '1');
13     g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

Milestone 4: Create onSubmit Client Script (Save Validation)

The screenshot shows the 'Catalog Client Scripts - New Record' form in ServiceNow. The form is titled 'Catalog Client Scripts - New Record' and has a 'Submit' button. A note at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The form fields are as follows:

- Name: Prevent save if Assigned To missing
- Applies to: A Catalog Item
- Active: ☒
- UI Type: Desktop
- Application: Global
- Type: onSubmit
- Catalog Item: (empty)
- Applies on a Catalog Item view: ☒
- Applies on Requested Items: ☐
- Applies on Catalog Tasks: ☐
- Applies on Target Record: ☐

The script editor contains the following code:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2
3 }
4
5 //Type appropriate comment here, and begin script below
6 function onSubmit() {
7   if (g_form.getValue('impact') == '1' &&
8       g_form.getValue('assigned_to') == '') {
9
10     g_form.showErrorMessage(
11       'Assigned To is mandatory for High impact incidents.'
12     );
13     return false;
14   }
15   return true;
16 }
17
18 }
```

Milestone 5: Create onCellEdit Client Script

The screenshot shows the 'Catalog Client Scripts - New Record' page in ServiceNow. The page has a dark blue header with the ServiceNow logo and navigation tabs. Below the header, there's a light blue banner with a warning message about client scripts running in strict mode. The main form is divided into two columns. The left column contains fields for 'Name' (Prevent state change via list edit), 'Applies to' (A Catalog Item), 'Active' (checked), and 'UI Type' (Desktop). The right column contains fields for 'Application' (Global), 'Type' (onChange), 'Catalog Item' (empty), and 'Variable name' (empty). Below these fields are checkboxes for 'Applies on a Catalog Item view' (checked), 'Applies on Requested Items' (unchecked), 'Applies on Catalog Tasks' (unchecked), and 'Applies on Target Record' (unchecked). At the bottom, there's a 'Script' editor with a code editor showing a JavaScript function. The function is named 'onCellEdit' and is triggered by the 'onChange' event. The code is as follows:

```
1 function onChange(control, oldValue, newValue, isLoading) {  
2  
3  
4  
5  
6 //Type appropriate comment here, and begin script below  
7 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
8  
9 alert('State cannot be updated using list editing. Please open the Incident.');10  
11 callback(false);  
12  
13 }  
14  
15 }
```

The bottom of the screen shows a Windows taskbar with various icons and a system tray showing the date and time as 02:04 PM on 30-09-2026.

Milestone 6: Testing the Configuration

The screenshot shows the 'Incident - Create INC0010002' form in ServiceNow. The page has a dark blue header with the ServiceNow logo and navigation tabs. Below the header, there's a light blue banner with a warning message about mandatory fields. The main form is divided into two columns. The left column contains fields for 'Number' (INC0010002), 'Caller' (System Administrator), and 'Watch list' (empty). The right column contains fields for 'Opened' (2026-09-30 01:36:14), 'Closed' (empty), 'Urgency' (1 - High), and 'State' (New). Below these fields is a 'Short description' field. Below the 'Short description' field is a 'Related Search Results' section with a search bar and a dropdown menu. The search bar is empty, and the dropdown menu is set to 'Knowledge & Catalog (All)'. Below the search bar, it says 'No results to display'. At the bottom, there's a 'Comments (Customer visible)' field. The bottom of the screen shows a Windows taskbar with various icons and a system tray showing the date and time as 02:06 PM on 30-09-2026.

Conclusion

SKILLWALLET

Dashboard

Skill Bank

Skill Wallet

Skill Card

Skill Credits

Dashboard > Skillwallet > Program > Servicenow system administrato... > Group-projects > Implement Client Script

0 AI Token300 Credits

SB

Implement Client Script & UI Policy (Incident)

TeamOverviewWorkspaceKanban

Conclusion

StoryDuration: 10m

Review

Description

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how **UI Policies** and **Client Scripts** can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like **Assigned To** and **Urgency** respond appropriately to changes in **Impact**, maintaining clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.

Resources

No resources available

Project Details

Project Progress90%

View Demo

View Repo

Milestone 1: Create UI Policy on Incident90%

Milestone 2: Create UI Policy Action – Urgency90%

Milestone 3: Create onChange Client Script90%

Milestone 4: Create onSubmit Client Script (Save Validation)90%

Milestone 5: Create onCellEdit Client Script90%

Milestone 6: Testing the Configuration90%

Conclusion90%

37°C
Mostly sunny

Search

hp

ENG IN

02:06 PM
30-09-2026